EF Core is based on, but distinct from, the existing Entity Framework libraries (currently up to version 6.x). It was built as part of the .NET Core push to work cross-platform, but with additional goals in mind. In particular, the EF Core team wanted to make a highly performant library that could be used with a wide range of databases.

There are many types of databases, but probably the most commonly used family is *relational* databases, accessed via Structured Query Language (SQL). This is the bread and butter of EF Core; it can map Microsoft SQL Server, SQLite, MySQL, Postgres, and many other relational databases. It even has a cool in-memory feature you can use when testing to create a temporary database. EF Core uses a provider model, so support for other relational databases can be plugged in later as they become available.

NOTE As of .NET Core 3.0, EF Core also works with nonrelational, NoSQL, or document databases like Cosmos DB, too. I'm going to consider mapping only to relational databases in this book, however, as that's the most common requirement in my experience. Historically, most data access, especially in the .NET ecosystem, has used relational databases, so it generally remains the most popular approach.

That discussion covers what EF Core is but doesn't dig into why you'd want to use it. Why not access the database directly by using the traditional ADO.NET libraries? Most of the arguments for using EF Core can be applied to ORMs in general, so what are the advantages of an ORM?

12.1.2 Why use an object-relational mapper?

One of the biggest advantages of an ORM is the speed with which it allows you to develop an application. You can stay in the familiar territory of